Universidad Mariano Gálvez de Guatemala
Sede El Naranjo
Facultad Ingeniería en Sistemas
Desarrollo Web - Sección B
Ing. JOSÉ MIGUEL VILLATORO HIDALGO

TutorGO

Marketplace educativo que conecta estudiantes con tutores

MANUAL TÉCNICO

Elaborado por

Erick Enrique González Canel	9490-20-2571

Control del documento

Campo	Detalle
Título	Manual técnico de TutorGO
Versión	1.0
Fecha	29 de octubre de 2026
Autor	Erick Enrique González Canel
Estado	Versión final para entrega

Versión	Fecha	Descripción del cambio
1.0	29/10/2026	Emisión inicial del manual técnico.

Tabla de contenido

1. Introducción....4
1.1 Propósito del documento....4
1.2 Alcance....4
1.3 Información del proyecto....4
1.4 Convenciones de este documento....4
2. Descripción general del sistema....5
2.1 Objetivo....5
2.2 Actores y roles....5
2.3 Módulos funcionales....5
2.4 Matriz de cumplimiento de los requerimientos....6
3. Arquitectura del sistema....8
3.1 Visión general....8
3.2 Tecnologías utilizadas....8
3.3 Canales de comunicación....9
3.4 Estructura del repositorio....10
4. Esquema conceptual de componentes....11
4.1 Capas del esquema....12
4.2 Flujo de estado y de datos....13
5. Descripción de componentes....14
5.1 Punto de entrada y configuración....14
5.2 Contextos....14
5.3 Guardas y elementos persistentes....14
5.4 Páginas....15
5.5 Componentes reutilizables y de dominio....16
5.6 Utilidades....17
6. Backend: API REST y tiempo real....18
6.1 Arranque del servidor....18
6.2 Middleware....18
6.3 Catálogo de endpoints....18
6.4 Reglas de negocio principales....20
6.5 Comunicación en tiempo real....21
7. Uso de TypeScript....23
7.1 Configuración....23
7.2 Técnicas de tipado empleadas....23
8. Uso de SASS/SCSS....26

8.1 Características de SCSS utilizadas 26
8.2 Bloques de estilo del proyecto 27
9. Modelo de datos: diagrama entidad-relación 29
9.1 Diagrama de relaciones 29
9.2 Diagrama con atributos: oferta, agenda y suscripciones 30
9.3 Diagrama con atributos: reseñas, mensajería y moderación 31
9.4 Descripción de las relaciones 31
9.5 Diccionario de datos 32
10. Seguridad 36
10.1 Autenticación híbrida: OAuth 2.0 y JWT 36
10.2 Control de la sesión 36
10.3 Autorización 37
10.4 Validación y protección de datos 37
11. Configuración, ejecución y despliegue 38
11.1 Variables de entorno 38
11.2 Ejecución en entorno local 38
11.3 Despliegue en la nube 38
12. Conclusiones 40
12.1 Posibles mejoras futuras 40
Anexo A. Glosario 41

1. Introducción

1.1 Propósito del documento

Este manual técnico describe la arquitectura, el diseño y la implementación de **TutorGO**, una plataforma web tipo *marketplace* educativo que conecta a estudiantes con tutores especializados. El documento está dirigido a quienes necesiten comprender, mantener, desplegar o ampliar el sistema, y reúne los elementos solicitados en el enunciado del proyecto de clase: el esquema conceptual de componentes, la descripción de componentes, la descripción del uso de TypeScript, la descripción del uso de SASS/SCSS y el diagrama entidad-relación.

1.2 Alcance

El manual cubre la aplicación completa en sus dos capas: el cliente (frontend) desarrollado con React y TypeScript, y el servidor (backend) desarrollado con Node.js, Express y TypeScript sobre una base de datos MongoDB. Incluye además los servicios externos que la plataforma integra (Google Identity, OpenStreetMap, OSRM), el mecanismo de comunicación en tiempo real, las medidas de seguridad y las instrucciones de configuración y despliegue en la nube.

1.3 Información del proyecto

Dato	Detalle
Nombre del sistema	TutorGO
Curso	Desarrollo Web
Institución	Universidad Mariano Gálvez de Guatemala, Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, sede El Naranjo
Autor	Erick Enrique González Canel (carné 9490-20-2571), desarrollo individual
Sitio funcional (frontend)	https://9490-20-2571-proyectofinaldw.netlify.app/
API (backend)	https://tutorgo-api-vqwv.onrender.com/ (verificación de estado en /api/salud)
Repositorio de código	https://github.com/ErickGonzalezUMG/ProyectoDW2026

1.4 Convenciones de este documento

Los nombres de archivos, rutas, funciones y variables se escriben con `tipografía de ancho fijo`. Las rutas de la API se indican con su método HTTP, por ejemplo `GET /api/buscar`. Los nombres de las colecciones de la base de datos se escriben en mayúsculas dentro de los diagramas (por ejemplo, TUTORIA) y como modelos de Mongoose en el código (por ejemplo, `Tutoria`). Por seguridad, el documento no incluye valores de credenciales, secretos ni cadenas de conexión; únicamente los nombres de las variables de entorno.

2. Descripción general del sistema

2.1 Objetivo

TutorGO resuelve de forma automatizada la logística de contratación de servicios educativos en modalidad virtual y presencial. La plataforma integra el cálculo dinámico de tarifas de traslado mediante mapas geoespaciales, la prevención estricta de traslapes en las agendas de tutores y estudiantes, las suscripciones periódicas por bloques de semanas, un chat en tiempo real y un módulo de moderación y auditoría de cuentas, todo con un diseño completamente responsivo para dispositivos móviles y de escritorio.

2.2 Actores y roles

Rol	Descripción	Capacidades principales
Estudiante	Persona que busca y contrata clases.	Buscar tutores con filtros geoespaciales. Agendar clases y suscripciones. Calificar clases realizadas. Denunciar conductas. Chatear con tutores.
Tutor	Persona que ofrece clases. Requiere aprobación del administrador para publicar ofertas.	Crear ofertas con modalidad y tarifas. Definir su disponibilidad semanal. Aprobar o rechazar solicitudes. Denunciar conductas. Chatear con estudiantes.
Administrador	Responsable de la operación y la moderación.	Gestionar categorías y subcategorías. Aprobar tutores. Resolver denuncias y apelaciones. Suspender y reactivar cuentas. Consultar la reportería analítica.
Usuario suspendido	Estado de una cuenta bloqueada por el administrador.	Puede iniciar sesión. Solo accede a la vista de Suspensión de Cuenta. Puede enviar una apelación formal.

2.3 Módulos funcionales

Módulo	Descripción	Componentes y rutas principales
Autenticación y onboarding	Inicio de sesión con Google, emisión de JWT y completado obligatorio del perfil.	`Login`, `Onboarding`, `POST /api/auth/google`, `PUT /api/usuarios/perfil`
Oferta académica	Creación y administración de ofertas por parte de tutores aprobados.	`MisOfertas`, `/api/ofertas`

Módulo	Descripción	Componentes y rutas principales
Búsqueda geoespacial	Filtrado de ofertas por cercanía, calificación, precio, categoría y modalidad; mapa y ruta.	`BuscarTutores`, `MapaBusqueda`, `GET /api/buscar`
Agendamiento	Disponibilidad semanal, solicitudes de clase, suscripciones y calendario.	`Disponibilidad`, `VentanaAgendar`, `Calendario`, `/api/tutorias`, `/api/suscripciones`
Mensajería	Chat privado en tiempo real con confirmación de lectura y marcas de tiempo.	`ChatFlotante`, `ChatContext`, `/api/chat`, Socket.io
Reseñas	Calificación de 1 a 5 estrellas y comentario tras una clase realizada.	`ResenaClase`, `/api/resenas`
Seguridad y moderación	Denuncias tipificadas, suspensión de cuentas y apelaciones.	`DenunciarClase`, `Suspendido`, `Moderacion`, `/api/denuncias`, `/api/apelaciones`, `/api/moderacion`
Administración y reportería	Catálogo de categorías, aprobación de tutores y analíticas con gráficos.	`Admin`, `AdminCategorias`, `AdminTutores`, `Analiticas`, `/api/analiticas`

2.4 Matriz de cumplimiento de los requerimientos

La siguiente matriz relaciona cada requerimiento del enunciado con su implementación en el sistema.

Requerimiento del enunciado	Implementación en TutorGO
Inicio de sesión y registro con Google (OAuth 2.0), captura de nombre, correo y fotografía.	El componente `Login` obtiene la credencial con `@react-oauth/google`; el backend la verifica con `google-auth-library` en `POST /api/auth/google` y extrae nombre, correo y fotografía.
Onboarding obligatorio: rol, teléfono, género, dirección con coordenadas en mapa, documento de identificación y afinidad de categorías; campos exclusivos del tutor.	Página `Onboarding` con `SelectorUbicacion` (Leaflet). Los tutores completan además hoja de vida (enlace a PDF o portafolio), biografía y tarifa base. Se guarda con `PUT /api/usuarios/perfil`.
Toda petición protegida valida el Bearer Token.	El middleware `validarToken` protege todas las rutas, salvo la emisión del token y la verificación de estado. El cliente añade el encabezado en la función central `peticion()`.
Ofertas de tutores aprobados con modalidad, tarifa por hora y tarifa por kilómetro.	Página `MisOfertas` y rutas `/api/ofertas` protegidas con `soloTutorAprobado`.
Filtros por radio, calificación, cantidad de reseñas, precio, categoría y modalidad.	`GET /api/buscar` aplica todos los filtros; el radio se calcula con la fórmula de Haversine.
Distancia real, mapa con la trayectoria y desglose del presupuesto.	Distancia por calles con OSRM, ruta dibujada con Leaflet en `MapaBusqueda` y desglose de tarifa y traslado en `VentanaAgendar`.

Requerimiento del enunciado	Implementación en TutorGO
Disponibilidad semanal y control estricto de traslapes para tutor y estudiante.	Cuadrícula en `Disponibilidad`; la función `validarHorario` rechaza solicitudes fuera de disponibilidad o que choquen con tutorías pendientes o aprobadas.
Calendario con vista diaria, semanal y mensual.	`Calendario` con FullCalendar (`timeGridDay`, `timeGridWeek`, `dayGridMonth`).
Suscripción periódica de 4 u 8 semanas con días y horas fijos.	`POST /api/suscripciones` genera las tutorías de cada semana validando los traslapes.
Chat en tiempo real con confirmación de lectura y marcas de tiempo.	Servidor Socket.io autenticado con JWT, modelo `Mensaje`, rutas `/api/chat` y la ventana flotante `ChatFlotante`.
Compartir el perfil del tutor por WhatsApp.	Botón en cada tarjeta de `BuscarTutores` que genera un enlace de WhatsApp con el resumen de la oferta, la tarifa y el enlace a la plataforma.
Calificaciones de 1 a 5 estrellas con comentario y promedio inmediato.	`ResenaClase` y `POST /api/resenas`; el promedio se calcula a partir de las reseñas almacenadas y queda disponible para auditoría.
Denuncias por inasistencia, conducta inapropiada o cobros indebidos con evidencia textual.	`DenunciarClase` con selector de tipo y descripción; modelo `Denuncia` con el campo `tipo`.
Cuenta suspendida: acceso solo a la vista de Suspensión con motivo, fecha, políticas y apelación.	Página `Suspendido`, guarda de rutas `RutaPrivada` y restricción en `validarToken` (solo permite `/api/auth/yo` y `/api/apelaciones`).
Panel del administrador: catálogo, moderación y reportería con gráficos.	Página `Admin` con las pestañas Categorías, Tutores, Moderación y Analíticas (Recharts).
Duración del JWT configurable, cierre de sesión forzado al expirar.	Variable de entorno `JWT_EXPIRES_IN`; ante un 401 el cliente ejecuta `LOGOUT`, marca `SESION_EXPIRADA` y redirige a `/login`.
React, SASS/SCSS, TypeScript, diseño responsivo y despliegue en la nube.	Frontend en Netlify, backend en Render y base de datos en MongoDB Atlas, con estilos en SCSS y todo el código en TypeScript.

3. Arquitectura del sistema

3.1 Visión general

TutorGO sigue una arquitectura cliente-servidor de tres capas. El cliente es una aplicación de página única (SPA) en React, servida como contenido estático. El servidor expone una API REST construida con Express y un servidor de WebSockets con Socket.io, ambos sobre el mismo proceso de Node.js. La persistencia se resuelve en MongoDB mediante Mongoose. Además, el sistema consume tres servicios externos: Google Identity para la autenticación, OpenStreetMap para las teselas del mapa y OSRM para el cálculo de rutas y distancias reales.

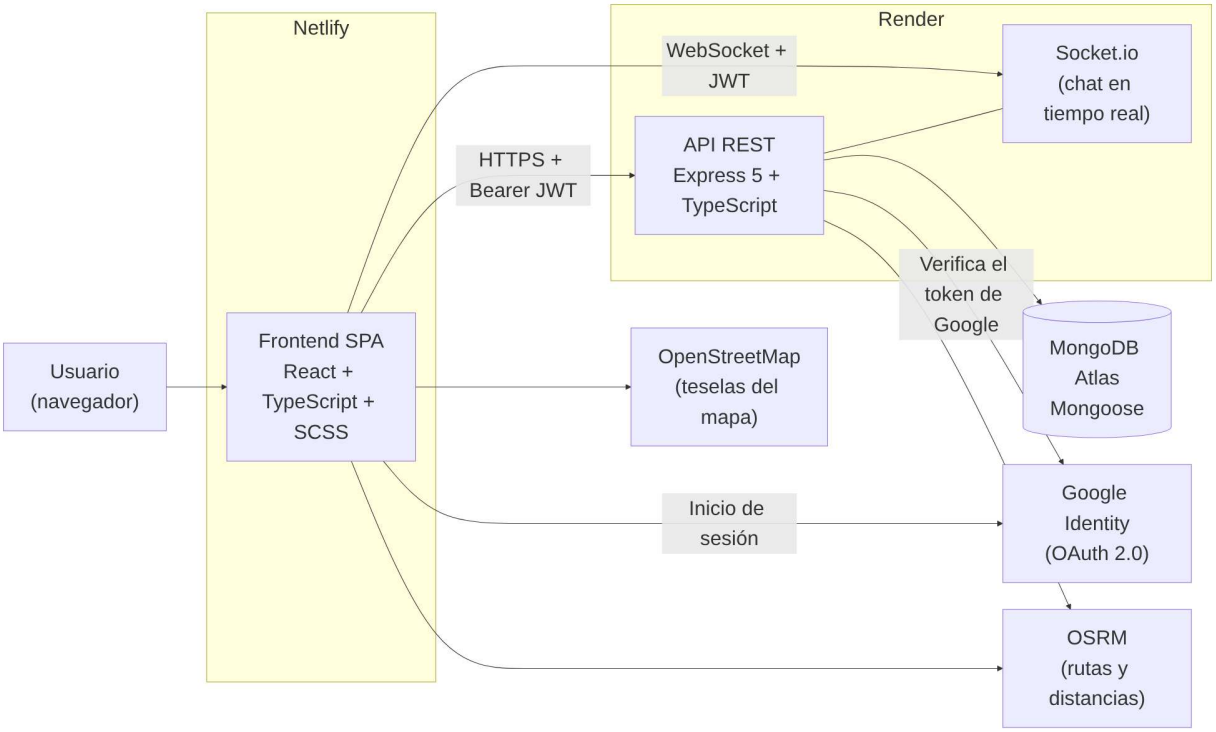


Figura 1. Arquitectura general de TutorGO y servicios externos.

3.2 Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Frontend	React 19 y TypeScript	Interfaz de usuario basada en componentes con tipado estático.
Frontend	Vite	Herramienta de construcción y servidor de desarrollo.
Frontend	react-router-dom 7	Enrutamiento y guardas de rutas privadas.
Frontend	Bootstrap 5.3, react-bootstrap 2 y Bootstrap Icons	Sistema de rejilla responsivo, componentes y iconografía.
Frontend	SASS/SCSS	Estilos propios con variables, mixins y anidamiento.

Capa	Tecnología	Uso en el proyecto
Frontend	Leaflet y react-leaflet 5	Mapas interactivos, marcadores, radio y ruta.
Frontend	FullCalendar 6	Calendario con vistas diaria, semanal y mensual.
Frontend	Recharts 3	Gráficos estadísticos de la reportería.
Frontend	@react-oauth/google	Botón e inicio de sesión con Google.
Frontend	socket.io-client 4	Conexión en tiempo real para el chat.
Backend	Node.js y Express 5	API REST modular.
Backend	TypeScript, tsx	Tipado estricto y ejecución en desarrollo con recarga automática.
Backend	Mongoose 9	Modelado de datos y acceso a MongoDB.
Backend	jsonwebtoken 9	Emisión y verificación de JWT.
Backend	google-auth-library	Verificación del token de identidad de Google.
Backend	Socket.io 4	Servidor de WebSockets autenticado.
Datos	MongoDB Atlas	Base de datos en la nube.
Despliegue	Netlify y Render	Hospedaje del frontend y del backend, con despliegue continuo desde GitHub.
Servicios externos	OpenStreetMap y OSRM	Teselas del mapa y cálculo de rutas por calles.

3.3 Canales de comunicación

- **API REST sobre HTTPS:** todas las operaciones de datos. Cada petición protegida incluye el encabezado `Authorization: Bearer <JWT>`.
- **WebSocket (Socket.io):** entrega inmediata de mensajes y de confirmaciones de lectura. La conexión se autentica enviando el JWT en el *handshake*.
- **Servicios de terceros:** el navegador solicita las teselas a OpenStreetMap y la ruta a OSRM; el backend usa OSRM para calcular la distancia real de traslado, y recurre a la distancia en línea recta si el servicio no responde.

3.4 Estructura del repositorio

```
ProyectoFinal/
├── backend/
│   └── src/
│       ├── index.ts            Arranque: Express, CORS, rutas y Socket.io
│       ├── sockets.ts          Servidor de WebSockets autenticado con JWT
│       ├── config/db.ts        Conexión a MongoDB
│       ├── middleware/         validarToken, permitirRoles, soloAdmin
│       ├── models/             11 modelos de Mongoose
│       ├── routes/             14 módulos de rutas de la API
│       └── utils/              agenda.ts (traslapes y precios), geo.ts (distancias)
├── frontend/
│   ├── public/_redirects       Reglas de enrutamiento de la SPA en Netlify
│   └── src/
│       ├── main.tsx            Punto de entrada y proveedores
│       ├── App.tsx             Rutas de la aplicación
│       ├── config.ts           URL de la API y nombre del sistema
│       ├── context/            AuthContext, ChatContext
│       ├── components/         Componentes reutilizables y de dominio
│       ├── pages/              Páginas asociadas a rutas
│       ├── types/index.ts      Tipos e interfaces compartidos
│       ├── utils/rutas.ts      Cálculo de rutas con OSRM
│       └── styles/main.scss    Estilos SCSS
└── README.md
```

4. Esquema conceptual de componentes

El frontend se organiza como un árbol de componentes React. En la raíz se encuentran los proveedores de contexto, que aportan el estado global de sesión y de chat. Debajo, el componente `App` define el enrutamiento y monta los elementos persistentes (barra de navegación, vigilante de sesión y ventana flotante de chat). Las páginas protegidas se envuelven con `RutaPrivada`, que decide si el usuario puede ver cada vista. Finalmente, las páginas se componen de componentes reutilizables y de dominio.

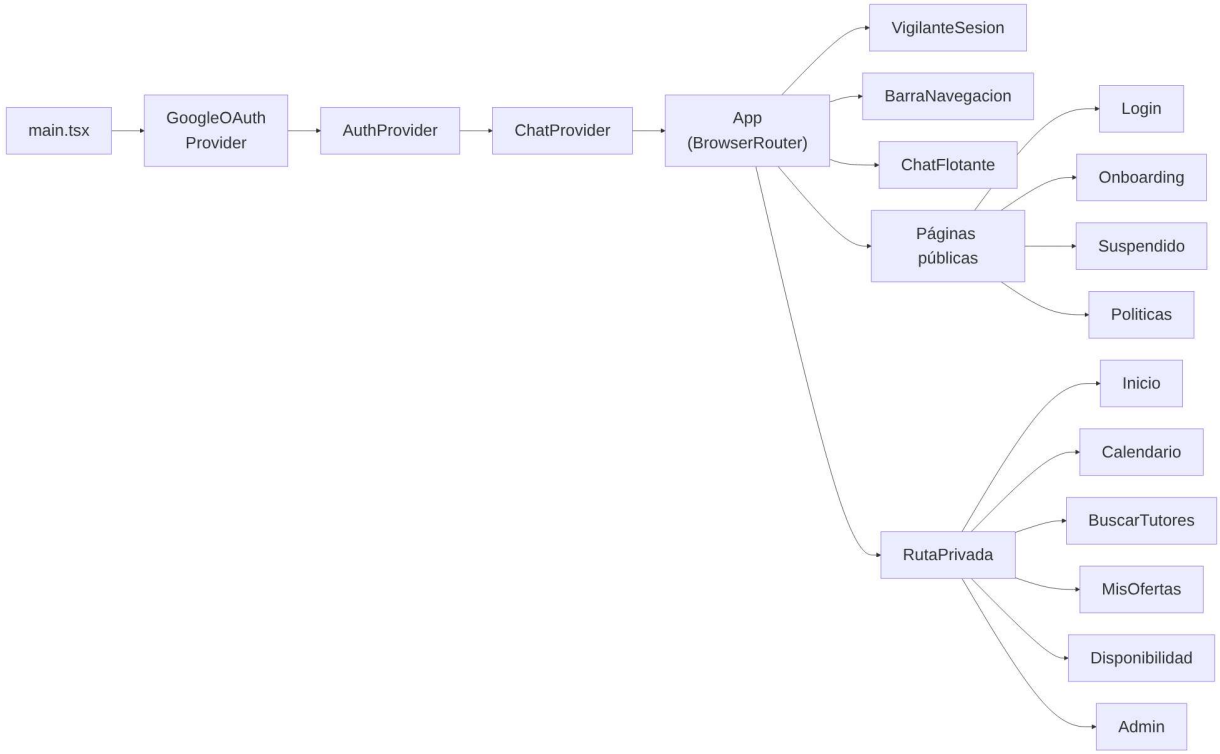


Figura 2. Esquema conceptual: proveedores, estructura de la aplicación y páginas.

El segundo esquema muestra, para cada página, los componentes que la componen y las relaciones de reutilización. Las líneas punteadas indican que la página invoca la apertura de `ChatFlotante` mediante `abrirChat()`.

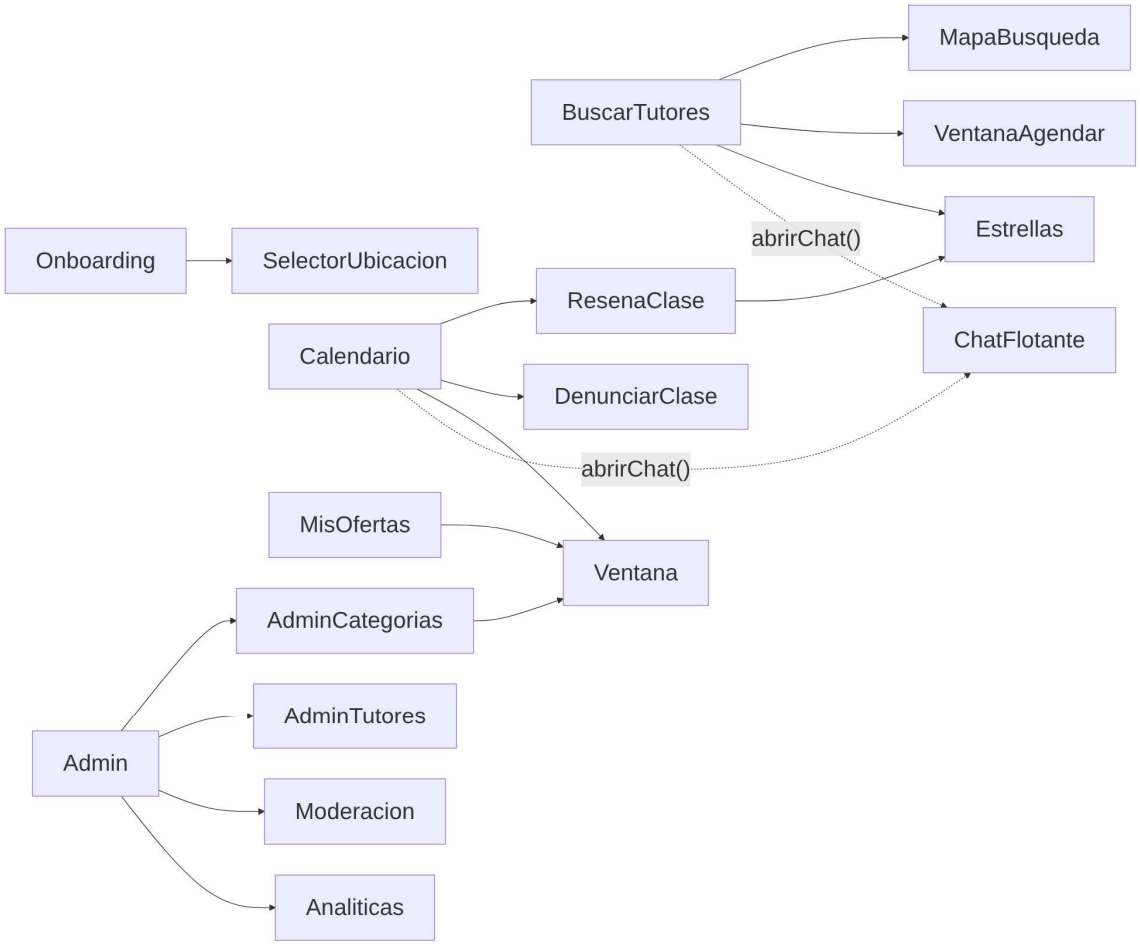


Figura 3. Esquema conceptual: componentes de dominio y reutilizables por página.

4.1 Capas del esquema

Capa	Elementos	Responsabilidad
Proveedores	`GoogleOAuthProvider`, `AuthProvider`, `ChatProvider`	Aportan el cliente de Google, el estado de sesión con la función `peticion()` y la conexión de chat.
Contenedor	`App` con `BrowserRouter`	Define las rutas y monta los elementos persistentes.
Elementos persistentes	`VigilanteSesion`, `BarraNavegacion`, `ChatFlotante`	Redirección al expirar la sesión, navegación por rol y mensajería disponible en todas las páginas.
Guarda de acceso	`RutaPrivada`	Aplica el orden de verificación: sesión, suspensión, onboarding y rol.
Páginas	`Login`, `Onboarding`, `Inicio`, `BuscarTutores`, `MisOfertas`, `Disponibilidad`, `Calendario`, `Admin`, `Suspendido`, `Politicas`	Cada una corresponde a una ruta de la aplicación.

Capa	Elementos	Responsabilidad
Componentes de dominio y reutilizables	`MapaBusqueda`, `VentanaAgendar`, `ResenaClase`, `DenunciarClase`, `Moderacion`, `Analiticas`, `Estrellas`, `Ventana`, entre otros	Encapsulan piezas de interfaz con una responsabilidad específica.

4.2 Flujo de estado y de datos

- **Estado global de sesión:** `AuthContext` usa `useReducer` con las acciones `LOGIN`, `LOGOUT`, `SESION_EXPIRADA` y `ACTUALIZAR_USUARIO`. La sesión se persiste en `localStorage` para sobrevivir a una recarga.
- **Acceso a la API:** los componentes nunca llaman a `fetch` directamente; usan `peticion(ruta, opciones)` de `AuthContext`, que añade el Bearer Token y, ante una respuesta 401, cierra la sesión y marca su expiración.
- **Estado de chat:** `ChatContext` mantiene la conexión Socket.io, el contador de mensajes sin leer y el control de la ventana flotante (abierta o cerrada y conversación activa). Cualquier página puede abrir el chat llamando a `abrirChat(idUsuario)`.
- **Estado local:** cada página gestiona sus filtros, formularios y listas con `useState` y `useEffect`.
- **Composición:** los datos fluyen de padres a hijos mediante *props* tipadas y los eventos suben mediante funciones de retorno (*callbacks*).

5. Descripción de componentes

Esta sección describe cada archivo del frontend según su función. Las tablas indican el archivo, la responsabilidad del componente y sus entradas (propiedades) o dependencias relevantes.

5.1 Punto de entrada y configuración

Elemento	Archivo	Descripción
Punto de entrada	`main.tsx`	Monta la aplicación envolviéndola con `GoogleOAuthProvider`, `AuthProvider` y `ChatProvider`, e importa Bootstrap, los iconos, los estilos de Leaflet y `main.scss`.
Aplicación	`App.tsx`	Declara las rutas con `react-router-dom`, aplica `RutaPrivada` con los roles permitidos en cada una y monta `VigilanteSesion`, `BarraNavegacion` y `ChatFlotante`.
Configuración	`config.ts`	Exporta la URL de la API (`VITE_API_URL`, con valor local por defecto) y el nombre del sistema.
Tipos compartidos	`types/index.ts`	Interfaces y tipos del dominio: `Usuario`, `Categoria`, `Oferta`, `ResultadoBusqueda`, `Tutoria`, `Resena`, `SuspensionInfo`, `ApelacionInfo`, entre otros.

5.2 Contextos

Contexto	Archivo	Descripción
`AuthContext`	`context/AuthContext.tsx`	Estado: `isAuthenticated`, `usuario`, `token`, `sesionExpirada`. Expone `estado`, `dispatch` y `peticion()`. Persiste token y usuario en `localStorage`. Hook de consumo: `useAuth()`.
`ChatContext`	`context/ChatContext.tsx`	Abre la conexión Socket.io con el JWT cuando el usuario es estudiante o tutor activo. Expone `socket`, `sinLeer`, `refrescarSinLeer()`, `abierto`, `conId`, `abrirChat()`, `cerrarChat()` y `elegirConversacion()`. Hook de consumo: `useChat()`.

5.3 Guardas y elementos persistentes

Componente	Archivo	Descripción
`RutaPrivada`	`components/RutaPrivada.tsx`	Propiedades: `children` y `roles?`. Verifica en orden: sesión activa (si no, `/login`), cuenta suspendida (`/suspendido`), perfil completo (`/onboarding`) y rol permitido (`/`).

Componente	Archivo	Descripción
VigilanteSesion	components/VigilanteSesion.tsx	No dibuja nada. Cuando sesionExpirada es verdadero, redirige a /login, cumpliendo el cierre forzado de sesión.
BarraNavegacion	components/BarraNavegacion.tsx	Barra colapsable (expand="md") con enlaces según el rol, foto y nombre del usuario y botón de cierre de sesión.
ChatFlotante	components/ChatFlotante.tsx	Botón circular fijo en la esquina inferior izquierda con insignia de mensajes sin leer. Abre una ventana con la lista de conversaciones y el historial, con hora y marcas de lectura (✓ enviado, ✓✓ leído). En celular ocupa toda la pantalla.

5.4 Páginas

Página	Archivo y ruta	Descripción
Login	pages/Login.tsx — /login	Botón de Google; envía la credencial a POST /api/auth/google y registra la sesión con la acción LOGIN.
Onboarding	pages/Onboarding.tsx — /onboarding	Formulario obligatorio de perfil: rol, teléfono, género, dirección con coordenadas (SelectorUbicacion), DPI y categorías; para tutores, hoja de vida, biografía y tarifa base.
Inicio	pages/Inicio.tsx — /	Pantalla de bienvenida con accesos directos según el rol.
BuscarTutores	pages/BuscarTutores.tsx — /buscar	Filtros (radio, calificación, reseñas, precio, categoría y modalidad), lista de ofertas, mapa con ruta, compartir por WhatsApp, botón de chat y apertura de VentanaAgendar.
MisOfertas	pages/MisOfertas.tsx — /ofertas	Alta, edición, activación y eliminación de ofertas del tutor aprobado.
Disponibilidad	pages/Disponibilidad.tsx — /disponibilidad	Cuadrícula semanal en la que el tutor marca sus bloques de disponibilidad por día y hora.
Calendario	pages/Calendario.tsx — /calendario	Calendario interactivo con vistas diaria, semanal y mensual; detalle de cada clase con aprobar, rechazar, cancelar, reseñar, denunciar y enviar mensaje. En celular inicia en vista diaria.
Admin	pages/Admin.tsx — /admin	Contenedor con pestañas: Categorías, Tutores, Moderación y Analíticas.
Suspendido	pages/Suspendido.tsx — /suspendido	Vista única de la cuenta bloqueada: motivo, fecha, enlace a las políticas y formulario de apelación con su estado.
Politicas	pages/Politicas.tsx — /politicas	Políticas de convivencia de la plataforma.

5.5 Componentes reutilizables y de dominio

Componente	Archivo	Descripción y propiedades
`Ventana`	`components/Ventana.tsx`	Modal genérico de confirmación o captura de texto. Props: `visible`, `titulo`, `mensaje?`, `conTexto?`, `valorInicial?`, `textoConfirmar?`, `peligro?`, `onConfirmar`, `onCerrar`.
`Estrellas`	`components/Estrellas.tsx`	Muestra la calificación promedio con estrellas. Props: `valor` y `cantidad?`; indica "Sin reseñas" cuando no hay calificaciones.
`MapaBusqueda`	`components/MapaBusqueda.tsx`	Mapa Leaflet con el origen, el círculo del radio, los marcadores de tutores y la ruta trazada. Props: `origen`, `radio`, `resultados`, `seleccionado`, `ruta?`, `onOrigen`, `onSeleccionar`.
`SelectorUbicacion`	`components/SelectorUbicacion.tsx`	Mapa en el que el usuario fija con un clic su dirección. Props: `latitud`, `longitud`, `onCambio`.
`VentanaAgendar`	`components/VentanaAgendar.tsx`	Modal para solicitar una clase única o una suscripción de 4 u 8 semanas; muestra los bloques libres y el desglose de tarifa y traslado. Props: `oferta`, `distanciaKm`, `onCerrar`.
`ResenaClase`	`components/ResenaClase.tsx`	Formulario de 1 a 5 estrellas y comentario para el estudiante tras una clase realizada. Prop: `tutoria`.
`DenunciarClase`	`components/DenunciarClase.tsx`	Formulario de denuncia con selector de tipo (inasistencia, conducta inapropiada, cobros indebidos) y descripción. Prop: `tutoria`.
`AdminCategorias`	`components/AdminCategorias.tsx`	CRUD de categorías y subcategorías con confirmación de eliminación.
`AdminTutores`	`components/AdminTutores.tsx`	Lista de tutores con acción para aprobarlos.
`Moderacion`	`components/Moderacion.tsx`	Pestañas Denuncias (con historial opcional y etiqueta de tipo), Apelaciones y Usuarios (historial de reportes por usuario, suspensión con motivo y reactivación).
`Analiticas`	`components/Analiticas.tsx`	Tarjetas de resumen, rankings de mejores y peores tutores con acceso a sus reseñas, gráfico de horas por categoría y gráfico de clases por estado con selector de mes o de año completo.

5.6 Utilidades

Elemento	Archivo	Descripción
`obtenerRuta`	`utils/rutas.ts`	Consulta a OSRM la ruta entre dos coordenadas y devuelve la distancia y la polilínea para dibujarla en el mapa.

6. Backend: API REST y tiempo real

6.1 Arranque del servidor

El archivo `src/index.ts` carga las variables de entorno, crea la aplicación Express, habilita CORS y el análisis de JSON, conecta con MongoDB y registra los módulos de rutas bajo el prefijo `/api`. Después crea un servidor HTTP con `createServer(app)`, inicializa Socket.io sobre ese mismo servidor mediante `iniciarSockets()` y lo pone a escuchar en el puerto definido en `PORT`.

```
const servidor = createServer(app)
iniciarSockets(servidor)

servidor.listen(PORT, () => {
  console.log(`Servidor escuchando en el puerto ${PORT}`)
})
```

6.2 Middleware

Middleware	Archivo	Función
`validarToken`	`middleware/validarToken.ts`	Exige el encabezado `Authorization: Bearer`, verifica la firma y la vigencia del JWT, comprueba que el usuario exista y consulta su estado en la base de datos. Responde 401 (`token_requerido`, `token_invalido`, `token_expirado`) o 403 (`cuenta_suspendida`) para suspendidos fuera de las rutas permitidas. Deja `usuarioId` y `usuarioRol` en la petición.
`permitirRoles`	`middleware/permitirRoles.ts`	Fábrica de middleware que restringe una ruta a uno o varios roles; responde 403 `no_permitido` en caso contrario.
`soloAdmin`	`middleware/soloAdmin.ts`	Restringe las rutas al rol administrador.
`soloTutorAprobado`	`routes/ofertas.ts`	Exige que el usuario sea tutor y haya sido aprobado por el administrador (`solo_tutores`, `tutor_no_aprobado`).

6.3 Catálogo de endpoints

Todas las rutas requieren JWT, con dos excepciones justificadas: `POST /api/auth/google`, que es la ruta que emite el token y por tanto no puede exigirlo, y `GET /api/salud`, la comprobación de estado que usa Render para verificar que el servicio está activo.

Método y ruta	Acceso	Descripción
`GET /api/salud`	Público	Comprobación de estado del servicio.
`POST /api/auth/google`	Público	Verifica la credencial de Google, crea el usuario si no existe y emite el JWT con los datos públicos.

Método y ruta	Acceso	Descripción
`GET /api/auth/yo`	Autenticado (incluye suspendidos)	Devuelve los datos actualizados del usuario de la sesión.
`PUT /api/usuarios/perfil`	Autenticado	Guarda el onboarding: rol, datos de contacto, ubicación, DPI, categorías y datos del tutor.
`GET /api/categorias`	Autenticado	Lista de categorías y subcategorías.
`POST`, `PUT`, `DELETE /api/categorias`	Administrador	Alta, edición y baja de categorías (se impide eliminar categorías con subcategorías).
`GET /api/admin/tutores`	Administrador	Lista de tutores con su estado de aprobación.
`PUT /api/admin/tutores/:id/aprobar`	Administrador	Aprueba a un tutor.
`GET /api/ofertas/mias`	Tutor, administrador	Ofertas del tutor autenticado.
`POST /api/ofertas`, `PUT /api/ofertas/:id`	Tutor aprobado	Crea o modifica una oferta con modalidad, tarifa por hora y tarifa por kilómetro.
`PUT /api/ofertas/:id/activa`, `DELETE /api/ofertas/:id`	Tutor aprobado	Activa o desactiva una oferta; la elimina.
`GET /api/disponibilidad/tutor/:id`	Autenticado	Bloques de disponibilidad semanal de un tutor.
`GET /api/disponibilidad/mia`, `PUT /api/disponibilidad`	Tutor, administrador	Consulta y reemplaza la disponibilidad propia.
`GET /api/buscar`	Estudiante, administrador	Búsqueda de ofertas con los filtros `lat`, `lng`, `radio`, `precioMin`, `precioMax`, `modalidad`, `categoria`, `calificacionMin` y `resenasMin`.
`POST /api/tutorias`	Estudiante, administrador	Solicita una clase; valida disponibilidad y traslapes y calcula el precio.
`GET /api/tutorias/mias`	Autenticado	Clases del usuario según su rol.
`PUT /api/tutorias/:id/estado`	Participantes y administrador	Aprueba, rechaza o cancela una clase según las reglas de transición.
`POST /api/suscripciones`	Estudiante, administrador	Crea una suscripción de 4 u 8 semanas y genera sus tutorías.
`GET /api/resenas/mias`, `POST /api/resenas`	Autenticado; calificar: estudiante	Consulta las reseñas del usuario; registra una calificación de 1 a 5 y un comentario.
`POST /api/denuncias`	Estudiante, tutor	Registra una denuncia con tipo y descripción sobre una clase realizada.
`GET /api/apelaciones/estado`, `POST /api/apelaciones`	Autenticado (incluye suspendidos)	Consulta la suspensión vigente y envía una apelación.

Método y ruta	Acceso	Descripción
`GET /api/moderacion/denuncias`, `PUT /api/moderacion/denuncias/:id`	Administrador	Bandeja de denuncias (con `estado=todas` y `usuario=ID`) y resolución (suspender o descartar).
`GET /api/moderacion/apelaciones`, `PUT /api/moderacion/apelaciones/:id`	Administrador	Bandeja de apelaciones; aceptar o rechazar.
`GET /api/moderacion/usuarios`	Administrador	Lista de usuarios con su estado.
`PUT /api/moderacion/usuarios/:id/suspender`, `.../reactivar`	Administrador	Suspensión con motivo y reactivación inmediata.
`GET /api/analiticas`	Administrador	Resumen, clases por estado (`mes` o `anio`), horas por categoría y rankings de tutores.
`GET /api/analiticas/tutores/:id/resenas`	Administrador	Reseñas de un tutor del ranking.
`GET /api/chat/conversaciones`, `GET /api/chat/sin-leer`	Estudiante, tutor	Lista de conversaciones con último mensaje y no leídos; total de mensajes sin leer.
`GET /api/chat/con/:id`, `PUT /api/chat/con/:id/leer`, `POST /api/chat/con/:id`	Estudiante, tutor	Historial con una persona (marca como leído), marcado de lectura y envío de un mensaje.

6.4 Reglas de negocio principales

6.4.1 Prevención de traslapes

La función `validarHorario` (`utils/agenda.ts`) se ejecuta antes de crear una tutoría. Verifica tres condiciones: que la clase esté contenida en un bloque de disponibilidad del tutor (`fuera_de_disponibilidad`), que el tutor no tenga otra clase pendiente o aprobada que se solape (`tutor_ocupado`) y que el estudiante tampoco la tenga (`estudiante_ocupado`). El solapamiento se detecta con la condición `inicio < fin_nuevo` y `fin > inicio_nuevo`.

6.4.2 Cálculo de tarifas y traslado

La función `precioSesion` calcula el total de una sesión con la fórmula siguiente. El traslado solo aplica en la modalidad de casa del estudiante; en las demás modalidades vale cero.

```
traslado = distanciaKm × tarifaKm      (solo si modalidad = casa_estudiante)
total    = tarifaHora × horas + traslado
```

La distancia se obtiene con `distanciaPorCalles` (`utils/geo.ts`), que consulta el servicio OSRM con un tiempo máximo de espera de 5 segundos. Si el servicio no responde, se utiliza la distancia en línea recta calculada con la fórmula de Haversine (`distanciaLinea`). En el frontend, `utils/rutas.ts` obtiene la polilínea para dibujar la trayectoria en el mapa.

6.4.3 Suscripciones

Una suscripción define 4 u 8 semanas, una fecha de inicio y una lista de sesiones fijas (día de la semana, hora de inicio y horas de duración). Al crearla, el servidor genera una tutoría por cada sesión y semana, validando cada una con las mismas reglas de traslapes y disponibilidad.

6.4.4 Clase realizada, reseñas y denuncias

Una clase se considera **realizada** cuando está aprobada (o completada) y su hora de fin ya pasó. Esta condición se deriva en el momento de la consulta y se aplica de forma uniforme en el calendario, las reseñas, las denuncias y las analíticas. Solo el estudiante de una clase realizada puede calificarla, una única vez; tanto el estudiante como el tutor pueden denunciar una clase realizada, una vez cada uno.

6.4.5 Moderación y suspensión

El administrador resuelve una denuncia suspendiendo al denunciado o descartándola. Al suspender se crea un registro de `Suspension` con el motivo y la cuenta cambia a estado suspendido. El usuario suspendido puede iniciar sesión, pero `validarToken` solo le permite acceder a `/api/auth/yo` y a `/api/apelaciones`. Si el administrador acepta su apelación, o reactiva la cuenta, esta vuelve a estado activo.

6.4.6 Zona horaria

Guatemala utiliza UTC-6 sin horario de verano. Las fechas se almacenan en UTC; las horas de las clases se interpretan con el desfase fijo de -06:00 y los rangos mensuales y anuales de las analíticas se calculan con ese mismo desfase.

6.5 Comunicación en tiempo real

El módulo `sockets.ts` inicializa Socket.io sobre el servidor HTTP. Antes de aceptar una conexión, un middleware valida el JWT enviado en el *handshake* (`auth.token`) con el mismo secreto que las rutas REST, verifica que el usuario exista y no esté suspendido, y rechaza la conexión en caso contrario. Cada usuario conectado se une a una sala personal `usuario:<id>`, de modo que el servidor puede dirigir eventos a una persona sin importar cuántas pestañas tenga abiertas. Además, la conexión se cierra automáticamente en el instante en que el token expira.

Evento	Dirección	Contenido y efecto
`mensaje`	Servidor → cliente	Mensaje nuevo (`_id`, `de`, `para`, `texto`, `leidoEn`, `createdAt`). Se envía al destinatario y también al emisor, para sincronizar sus demás pestañas.
`leidos`	Servidor → cliente	Indica quién (`por`) leyó los mensajes del receptor; el emisor actualiza sus marcas de ✓ a ✓✓.

El envío de mensajes se realiza por la ruta REST `POST /api/chat/con/:id`, lo que permite reutilizar las validaciones y la autenticación; el servidor persiste el mensaje en la colección `Mensaje` y lo empuja en tiempo real por Socket.io. Las reglas de acceso son: el chat solo existe entre un estudiante y un tutor; el estudiante puede escribir a cualquier tutor aprobado; el tutor puede responder a quien le escribió o tenga una clase con él; los administradores y los usuarios suspendidos quedan excluidos.

7. Uso de TypeScript

Todo el código del proyecto, tanto del frontend como del backend, está escrito en TypeScript. El tipado estático permite detectar en tiempo de compilación errores de nombres, de tipos de datos y de contratos entre capas, y documenta la intención del código. La compilación de TypeScript forma parte del proceso de construcción: el comando `npm run build` del backend ejecuta `tsc`, de manera que un error de tipos impide el despliegue.

7.1 Configuración

Proyecto	Configuración relevante
Backend (`tsconfig.json`)	`target: ES2022`, `module: commonjs`, `strict: true`, `esModuleInterop: true`, `rootDir: src`, `outDir: dist`.
Frontend (`tsconfig.app.json`)	`target: es2023`, `moduleResolution: bundler`, `jsx: react-jsx`, `verbatimModuleSyntax`, `noUnusedLocals`, `noUnusedParameters`, `erasableSyntaxOnly`, `noFallthroughCasesInSwitch`.

7.2 Técnicas de tipado empleadas

Interfaces del dominio compartidas

El archivo `types/index.ts` centraliza las interfaces que describen los datos que viajan entre la API y los componentes, de modo que cualquier cambio en un contrato se detecta en todos los lugares donde se usa.

```
export type Modalidad = 'virtual' | 'casa_tutor' | 'casa_estudiante'
export type EstadoTutoria = 'pendiente' | 'aprobada' | 'rechazada' | 'cancelada' | 'completada'

export interface Tutoria {
  _id: string
  oferta: { _id: string; titulo: string; modalidad: Modalidad } | null
  tutor: { _id: string; nombre: string }
  estudiante: { _id: string; nombre: string }
  inicio: string
  fin: string
  estado: EstadoTutoria
  total: number
}
```

Tipos de unión literal y mapas tipados

Los estados y las modalidades se modelan como uniones de cadenas literales, que el compilador valida en cada asignación. Los diccionarios de textos se tipan con `Record<K, V>`, lo que obliga a cubrir todas las claves posibles, por ejemplo las etiquetas de `EstadoTutoria` en el calendario.

Propiedades tipadas en componentes

Cada componente declara una interfaz `Props`. Se usa además el acceso indexado de tipos (`Usuario['rol'][]`) para reutilizar el tipo del rol sin duplicarlo.

```
interface Props {
  children: ReactNode
  roles?: Usuario['rol'][]
}

function RutaPrivada({ children, roles }: Props) { ... }
```

Uniones discriminadas en el estado global

El reductor de `AuthContext` recibe una unión discriminada de acciones. TypeScript restringe el contenido de cada acción según su campo `type`, y evita enviar acciones mal formadas.

```
type Accion =
  | { type: 'LOGIN'; usuario: Usuario; token: string }
  | { type: 'LOGOUT' }
  | { type: 'SESION_EXPIRADA' }
  | { type: 'ACTUALIZAR_USUARIO'; usuario: Usuario }
```

Extensión de tipos de Express

En el backend, la interfaz `RequestAutenticado` extiende `Request` para añadir los datos que deja el middleware de autenticación, y `PayloadToken` describe el contenido del JWT.

```
export interface PayloadToken {
  id: string
  rol: 'estudiante' | 'tutor' | 'admin'
}

export interface RequestAutenticado extends Request {
  usuarioId?: string
  usuarioRol?: 'estudiante' | 'tutor' | 'admin'
}
```

Guardas de tipo

La función `esError` de `utils/agenda.ts` es una guarda de tipo (*type predicate*): permite distinguir en una sola comprobación si una función devolvió un resultado válido o un error tipado, y el compilador refina el tipo en cada rama.

```
export function esError(valor: unknown): valor is ErrorAgenda {
  const v = valor as ErrorAgenda
  return typeof valor === 'object' && valor !== null &&
    v.error !== undefined && v.status !== undefined
}
```

Modelos de Mongoose tipados

Cada modelo declara una interfaz del documento y la pasa como genérico al esquema y al modelo, de modo que las consultas devuelven datos con el tipo correcto.

```
export interface IDenuncia {
  tutoria: Types.ObjectId
  denunciante: Types.ObjectId
  denunciado: Types.ObjectId
  tipo?: TipoDenuncia
  motivo: string
  estado: EstadoDenuncia
}

const denunciaSchema = new Schema<IDenuncia>({ ... }, { timestamps: true })
export default model<IDenuncia>('Denuncia', denunciaSchema)
```

Genéricos, unknown y estrechamiento de tipos

- Genéricos en estado y promesas: `useState<string>('')`, `Promise<void>`, `Promise<Response>`.
- Datos externos tratados como `unknown` y validados antes de usarse, por ejemplo `typeof d.motivo === 'string'` antes de aplicar `trim()`.
- Conversiones explícitas y acotadas de las respuestas JSON, como `(await r.json()) as { denuncias: DenunciaAdmin[] }`.
- Importaciones solo de tipos con `import type`, exigidas por `verbatimModuleSyntax`.
- Tipos de retorno explícitos en los controladores (`Promise<void>`) para evitar respuestas olvidadas.

8. Uso de SASS/SCSS

Los estilos propios de la aplicación están escritos en SCSS en el archivo `src/styles/main.scss`, que se importa en `main.tsx` y es compilado por Vite. SCSS se combina con Bootstrap 5 (que aporta la rejilla responsiva y los componentes base): Bootstrap define la estructura y `main.scss` aplica la identidad visual de TutorGO y sobrescribe los estilos por defecto.

8.1 Características de SCSS utilizadas

Variables

La paleta, el radio de los bordes y la sombra se definen una sola vez y se reutilizan en todo el archivo; para cambiar la identidad visual basta con modificar estas variables.

```
$color-principal: #1f4e79;
$color-secundario: #2e86c1;
$color-fondo: #f4f6fb;
$color-texto: #1e293b;
$radio: 12px;
$sombra: 0 4px 16px rgba(15, 23, 42, 0.08);
```

Mixins

El *mixin* `tarjeta` agrupa el estilo común de las tarjetas (fondo, borde, radio y sombra) y se incluye con `@include` en las clases `.card`, `.tarjeta` y `.hero`, evitando repetir código.

```
@mixin tarjeta {
  background: #fff;
  border: none;
  border-radius: $radio;
  box-shadow: $sombra;
}

.card,
.tarjeta {
  @include tarjeta;
}
```

Anidamiento y selector padre

Los estilos se anidan siguiendo la estructura del HTML y se usa `&` para los estados, lo que los vuelve más legibles que el CSS plano.

```
.barra-principal {
  background: linear-gradient(90deg, $color-principal, $color-secundario);

  .nav-link {
    opacity: 0.85;

    &:hover {
      opacity: 1;
    }
  }
}
```

Módulos y funciones de color

Con `@use "sass:color"` se importa el módulo de color de Sass. La función `color.adjust` calcula variantes de un color a partir de las variables, por ejemplo el tono oscurecido de los botones al pasar el cursor o el tono claro de las celdas de disponibilidad.

```
@use "sass:color";

.btn-primary {
  background-color: $color-principal;

  &:hover {
    background-color: color.adjust($color-principal, $lightness: -8%);
  }
}
```

Diseño responsivo con media queries

Además de las clases de rejilla de Bootstrap, `main.scss` define reglas con `@media` para pantallas menores de 768 px (ajuste de la barra del calendario) y, anidada dentro de `.chat-panel`, una regla para pantallas menores de 576 px que convierte la ventana de chat en una vista de pantalla completa.

```
.chat-panel {
  position: fixed;
  width: 360px;
  height: 500px;

  @media (max-width: 575.98px) {
    inset: 0;
    width: auto;
    height: auto;
    border-radius: 0;
  }
}
```

8.2 Bloques de estilo del proyecto

Clase o bloque	Propósito
`.barra-principal`	Barra de navegación con degradado, marca, enlaces y avatar.
`.card`, `.tarjeta`, `.hero`	Tarjetas y encabezados con el mixin `tarjeta`.
`.titulo-seccion`, `.icono-marca`	Títulos y detalles tipográficos con la paleta de la marca.
`.btn-primary`, `.nav-tabs`, `.list-group`	Sobrescritura de componentes de Bootstrap.
`.tabla-disponibilidad`	Cuadrícula de disponibilidad del tutor con celdas activas y efecto al pasar el cursor.
`.pestanas-scroll`	Pestañas que se desplazan en horizontal en pantallas angostas.
`.chat-fab`, `.chat-panel`	Botón flotante, ventana, burbujas de mensajes y marcas de lectura del chat.
`@media (max-width: 767.98px)`	Ajustes para celulares: barra del calendario y relleno de tarjetas.

9. Modelo de datos: diagrama entidad-relación

La base de datos es MongoDB y el acceso se hace con Mongoose. El modelo contiene once colecciones. Las relaciones entre colecciones se implementan con referencias por `ObjectId` (claves foráneas lógicas), y las restricciones de integridad se refuerzan con índices únicos y validaciones del esquema.

9.1 Diagrama de relaciones

El primer diagrama presenta las entidades y las cardinalidades de sus relaciones. Los diagramas siguientes detallan los atributos de cada entidad.

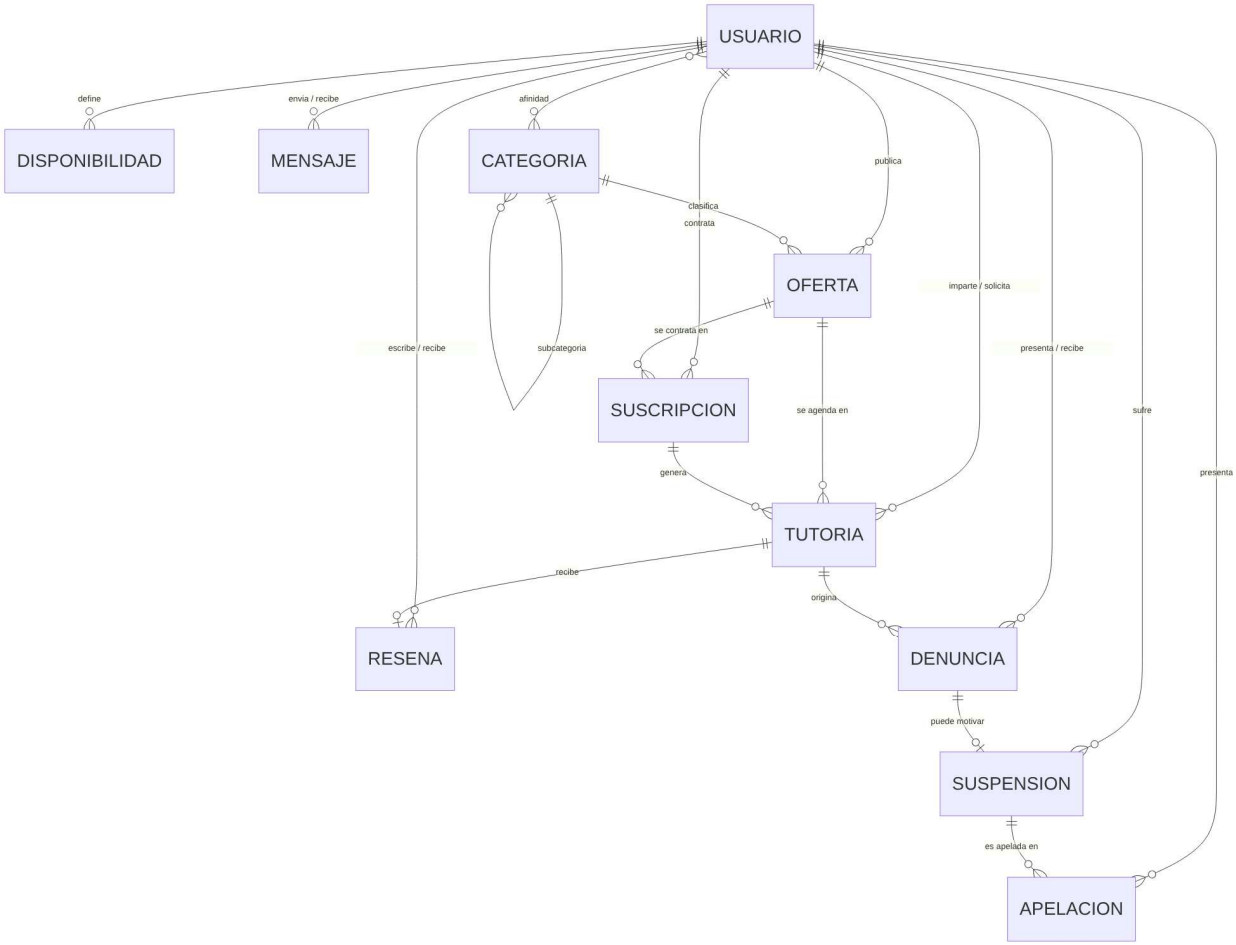


Figura 4. Diagrama entidad-relación: entidades y cardinalidades.

9.2 Diagrama con atributos: oferta, agenda y suscripciones

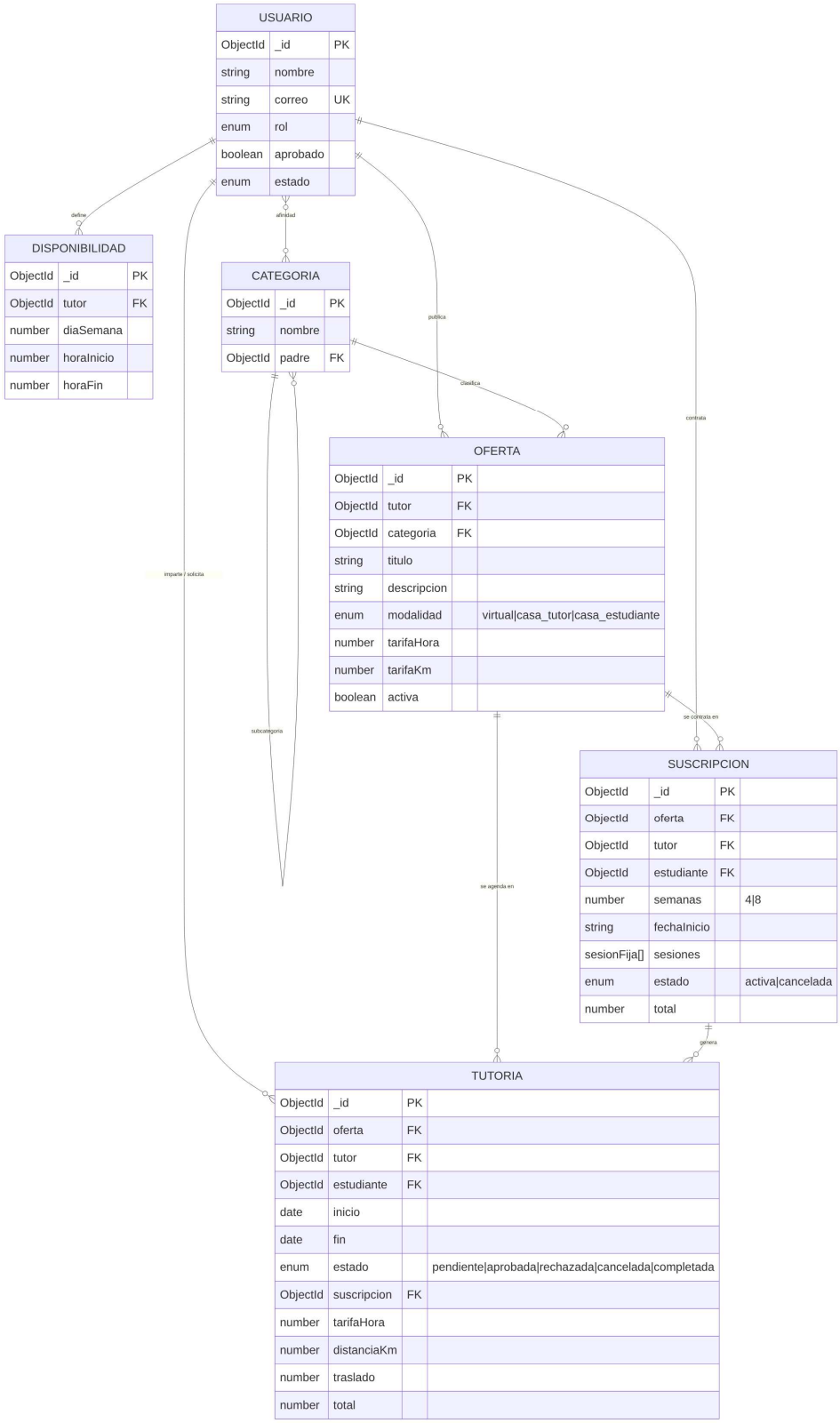


Figura 5. Entidades de catálogo, oferta, disponibilidad, tutoría y suscripción (USUARIO resumido).

9.3 Diagrama con atributos: reseñas, mensajería y moderación

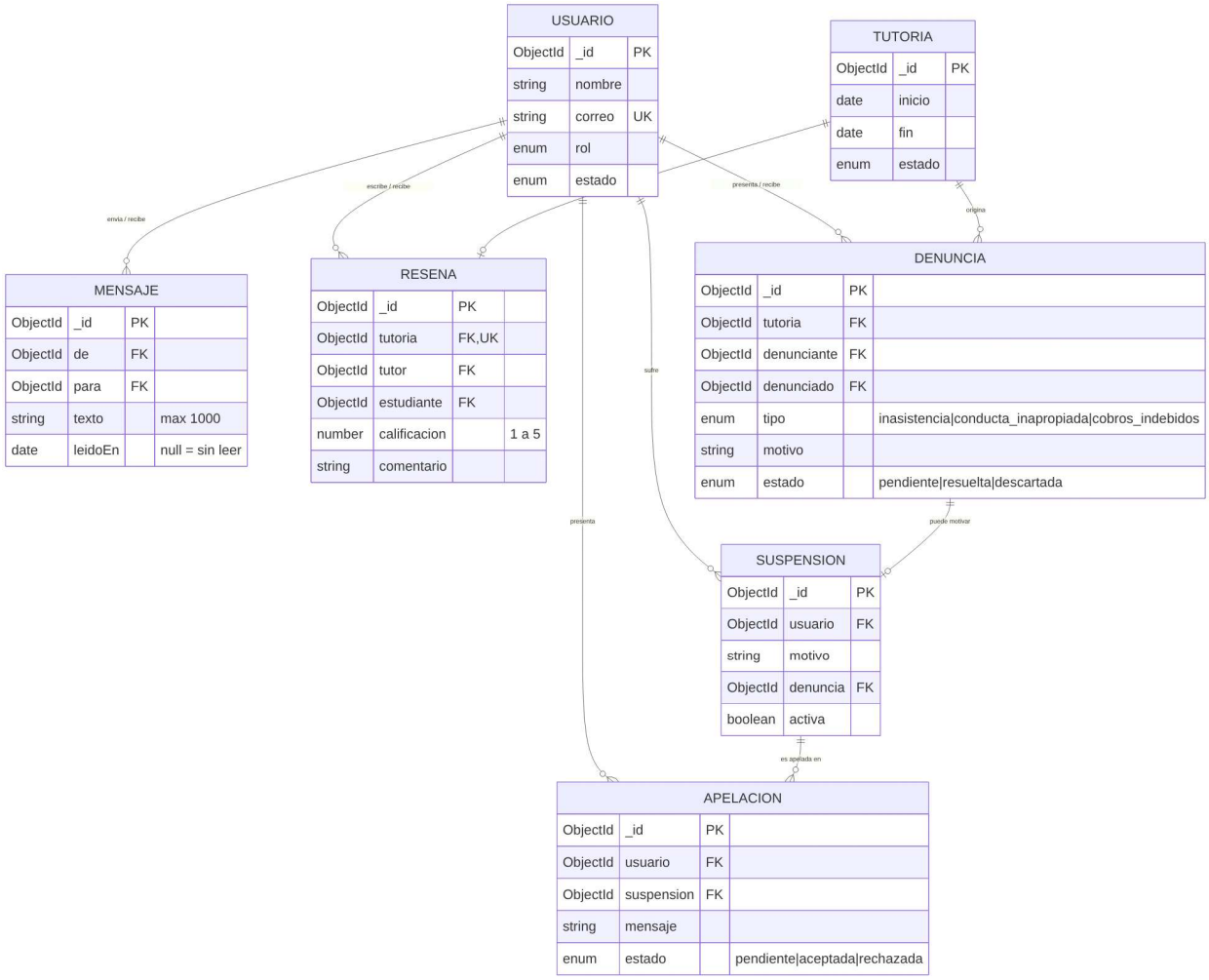


Figura 6. Entidades de reseñas, mensajes, denuncias, suspensiones y apelaciones.

9.4 Descripción de las relaciones

Relación	Cardinalidad	Descripción
USUARIO – OFERTA	1 a N	Un tutor publica varias ofertas.
USUARIO – DISPONIBILIDAD	1 a N	Un tutor define varios bloques semanales.
USUARIO – TUTORIA	1 a N (dos veces)	Un usuario participa como tutor y otro como estudiante en cada tutoría.
USUARIO – SUSCRIPCION	1 a N	Un estudiante contrata varias suscripciones.
USUARIO – CATEGORIA	N a M	Las categorías de afinidad del usuario se guardan como una lista de referencias.
CATEGORIA – CATEGORIA	1 a N	Una categoría puede tener subcategorías mediante el campo `padre`.

Relación	Cardinalidad	Descripción
CATEGORIA – OFERTA	1 a N	Cada oferta pertenece a una categoría.
OFERTA – TUTORIA / SUSCRIPCION	1 a N	Las clases y las suscripciones se agendan sobre una oferta.
SUSCRIPCION – TUTORIA	1 a N	Una suscripción genera una tutoría por cada sesión y semana.
TUTORIA – RESENA	1 a 0..1	Cada clase recibe a lo sumo una reseña (índice único).
TUTORIA – DENUNCIA	1 a N	Una clase puede originar denuncias, una por denunciante (índice único).
DENUNCIA – SUSPENSION	1 a 0..1	Una denuncia resuelta puede motivar una suspensión.
SUSPENSION – APELACION	1 a N	Una suspensión puede ser apelada por el usuario.
USUARIO – MENSAJE	1 a N (dos veces)	Un usuario envía y recibe mensajes.

9.5 Diccionario de datos

Todas las colecciones incluyen además el identificador `_id` y las marcas de tiempo `createdAt` y `updatedAt` generadas por Mongoose (`timestamps: true`), salvo que se indique lo contrario.

USUARIO

Cuenta de una persona en la plataforma, creada al iniciar sesión con Google.

Campo	Tipo	Restricciones y descripción
nombre, correo, foto	string	Capturados de la identidad de Google. `correo` es único.
rol	enum	`estudiante`, `tutor` o `admin`.
perfilCompleto	boolean	Indica si el usuario terminó el onboarding.
telefono, genero, direccion	string	Datos complementarios del onboarding.
latitud, longitud	number	Coordenadas de la dirección habitual fijadas en el mapa.
dpi	string	Documento de identificación personal.
categorias	ObjectId[]	Referencias a las categorías de afinidad.
hojaVida, biografia, tarifaBase	string, string, number	Campos exclusivos del tutor: enlace a la hoja de vida o portafolio, biografía profesional y tarifa base por hora.
aprobado	boolean	Aprobación del tutor por el administrador.
estado	enum	`activo` o `suspendido`.

CATEGORIA

Catálogo jerárquico de áreas de tutoría.

Campo	Tipo	Restricciones y descripción
nombre	string	Nombre de la categoría.
padre	ObjectId	Referencia a la categoría superior; nulo si es de primer nivel.

OFERTA

Servicio de tutoría publicado por un tutor aprobado.

Campo	Tipo	Restricciones y descripción
tutor	ObjectId	Referencia a `USUARIO`.
categoria	ObjectId	Referencia a `CATEGORIA`.
titulo, descripcion	string	Presentación de la oferta.
modalidad	enum	`virtual`, `casa_tutor` o `casa_estudiante`.
tarifaHora	number	Tarifa base por hora de clase.
tarifaKm	number	Tarifa adicional por kilómetro de traslado.
activa	boolean	Si la oferta está visible en las búsquedas.

DISPONIBILIDAD

Bloque semanal en el que un tutor atiende clases (colección `disponibilidades`).

Campo	Tipo	Restricciones y descripción
tutor	ObjectId	Referencia a `USUARIO`.
diaSemana	number	De 0 (domingo) a 6 (sábado).
horaInicio, horaFin	number	Horas del bloque.

TUTORIA

Clase agendada entre un estudiante y un tutor.

Campo	Tipo	Restricciones y descripción
oferta, tutor, estudiante	ObjectId	Referencias a `OFERTA` y a `USUARIO`.
inicio, fin	date	Horario de la clase.
estado	enum	`pendiente`, `aprobada`, `rechazada`, `cancelada` o `completada`.

Campo	Tipo	Restricciones y descripción
suscripcion	ObjectId	Referencia opcional a la suscripción que la generó.
tarifaHora, distanciaKm, traslado, total	number	Desglose del presupuesto calculado al momento de agendar.

SUSCRIPCION

Contratación recurrente por bloques de semanas.

Campo	Tipo	Restricciones y descripción
oferta, tutor, estudiante	ObjectId	Referencias.
semanas	number	4 u 8.
fechaInicio	string	Fecha de inicio en formato AAAA-MM-DD.
sesiones	sesionFija[]	Lista de sesiones fijas: día de la semana, hora de inicio y horas de duración.
estado	enum	`activa` o `cancelada`.
total	number	Costo total de la suscripción.

RESENA

Calificación de una clase realizada.

Campo	Tipo	Restricciones y descripción
tutoria	ObjectId	Referencia a `TUTORIA`; índice único (una reseña por clase).
tutor, estudiante	ObjectId	Referencias a `USUARIO`.
calificacion	number	Entero de 1 a 5.
comentario	string	Comentario cualitativo de hasta 500 caracteres.

DENUNCIA

Reporte de conducta presentado por un participante de una clase.

Campo	Tipo	Restricciones y descripción
tutoria	ObjectId	Referencia a `TUTORIA`. Índice único con `denunciante`.
denunciante, denunciado	ObjectId	Referencias a `USUARIO`.
tipo	enum	`inasistencia`, `conducta_inapropiada` o `cobros_indebidos`.
motivo	string	Descripción y evidencia textual, hasta 500 caracteres.
estado	enum	`pendiente`, `resuelta` o `descartada`.

SUSPENSION

Bloqueo de una cuenta.

Campo	Tipo	Restricciones y descripción
usuario	ObjectId	Cuenta suspendida.
motivo	string	Razón del bloqueo, visible para el usuario.
denuncia	ObjectId	Denuncia de origen; nulo si fue una suspensión directa.
activa	boolean	Si la suspensión sigue vigente.

APELACION

Solicitud formal del usuario suspendido.

Campo	Tipo	Restricciones y descripción
usuario, suspension	ObjectId	Referencias.
mensaje	string	Exposición del caso ante el administrador.
estado	enum	`pendiente`, `aceptada` o `rechazada`.

MENSAJE

Mensaje de chat entre un estudiante y un tutor.

Campo	Tipo	Restricciones y descripción
de, para	ObjectId	Emisor y receptor (referencias a `USUARIO`).
texto	string	Contenido, de 1 a 1000 caracteres.
leidoEn	date	Momento de lectura; nulo mientras no se ha leído (confirmación de lectura).

10. Seguridad

10.1 Autenticación híbrida: OAuth 2.0 y JWT

La autenticación combina el inicio de sesión con Google (OAuth 2.0) y la emisión de un token JWT propio. El navegador obtiene el token de identidad de Google; el backend lo verifica con `google-auth-library` contra el identificador de cliente configurado, crea la cuenta si es la primera vez y firma un JWT con los datos mínimos (`id` y `rol`). Desde ese momento, todas las peticiones se autentican con el JWT.

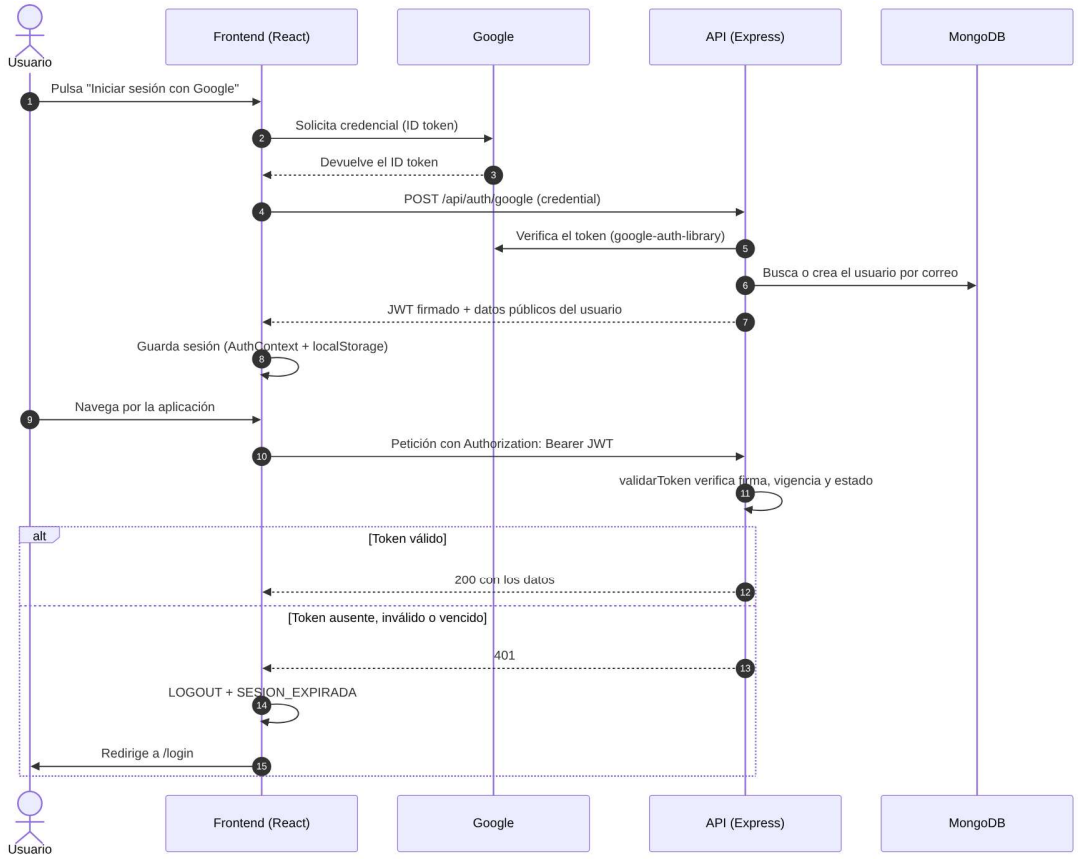


Figura 7. Flujo de autenticación y manejo de la expiración del token.

10.2 Control de la sesión

- **Validación en cada petición:** `validarToken` verifica la firma y la vigencia del token y consulta el estado actual del usuario en la base de datos; así, una suspensión surte efecto de inmediato sin esperar a que el token expire.
- **Duración configurable:** el tiempo de vida del token se define en la variable de entorno `JWT_EXPIRES_IN` del backend (por ejemplo `5m` o `1h`); si no se define se usa 5 minutos.
- **Cierre de sesión forzado:** cuando la API responde 401, la función `peticion()` ejecuta `LOGOUT` y `SESION_EXPIRADA`, y `VigilanteSesion` redirige al inicio de sesión.

- **Token en cada petición:** el frontend no usa `fetch` directamente para recursos protegidos; todas las llamadas pasan por `peticion()`, que añade el encabezado `Authorization`.
- **WebSocket autenticado:** la conexión de chat exige el JWT en el *handshake* y se cierra al vencer el token.

10.3 Autorización

- **Por rol:** `permitirRoles`, `soloAdmin` y `soloTutorAprobado` restringen cada ruta. En el cliente, `RutaPrivada` oculta las páginas que no corresponden al rol.
- **Por propiedad:** las rutas verifican que el usuario sea participante del recurso (por ejemplo, solo quienes participaron en una clase pueden denunciarla o cambiar su estado).
- **Cuentas suspendidas:** `validarToken` solo permite `/api/auth/yo` y `/api/apelaciones`; el resto responde 403 `cuenta_suspendida`, y el cliente redirige a la vista de Suspensión. Los administradores no pueden ser suspendidos.

10.4 Validación y protección de datos

- Validación de los datos de entrada en cada ruta (tipos, rangos y longitudes) con códigos de error descriptivos.
- Índices únicos que impiden duplicados: una reseña por clase, una denuncia por denunciante y clase, y un correo por usuario.
- Los secretos (`JWT_SECRET`, `MONGO_URI`, `GOOGLE_CLIENT_ID`) se leen exclusivamente de variables de entorno y nunca se incluyen en el repositorio; el archivo `.env` está excluido del control de versiones.
- Comunicación cifrada: tanto Netlify como Render sirven el sitio y la API por HTTPS.

> **Mejora recomendada:** actualmente CORS y Socket.io aceptan cualquier origen, lo cual es válido porque la API exige JWT en cada petición. Como refuerzo adicional se puede restringir el origen permitido al dominio de Netlify mediante una variable de entorno.

11. Configuración, ejecución y despliegue

11.1 Variables de entorno

Ninguna variable contiene valores en este documento. Cada una se define en el archivo `.env` local (existe un `.env.example` como plantilla) y en el panel de la plataforma de despliegue.

Proyecto	Variable	Descripción
Backend	`PORT`	Puerto de escucha del servidor (Render lo asigna automáticamente).
Backend	`MONGO_URI`	Cadena de conexión a MongoDB Atlas.
Backend	`JWT_SECRET`	Secreto con el que se firman y verifican los JWT.
Backend	`JWT_EXPIRES_IN`	Duración del token (por ejemplo `5m`, `1h`).
Backend	`GOOGLE_CLIENT_ID`	Identificador de cliente de OAuth de Google.
Frontend	`VITE_API_URL`	URL base de la API; si no se define, se usa `http://localhost:3001`.
Frontend	`VITE_GOOGLE_CLIENT_ID`	Identificador de cliente de OAuth de Google para el botón de inicio de sesión.

11.2 Ejecución en entorno local

1. Clonar el repositorio e ingresar a la carpeta del proyecto.
2. En `backend`: crear el archivo `.env` a partir de `.env.example`, ejecutar `npm install` y luego `npm run dev` (recarga automática con `tsx watch`).
3. En `frontend`: crear el archivo `.env` a partir de `.env.example`, ejecutar `npm install` y luego `npm run dev` (servidor de Vite).
4. Abrir la dirección que muestra Vite en el navegador. En la consola de Google Cloud, el origen local debe estar autorizado para el identificador de cliente.

11.3 Despliegue en la nube

Componente	Plataforma	Configuración
Frontend	Netlify	Construcción con `npm run build` y publicación de la carpeta `dist`. El archivo `public/_redirects` contiene `/* /index.html 200`, necesario para que las rutas de la SPA funcionen al recargar. Variables `VITE_*` definidas en el panel.

Componente	Plataforma	Configuración
Backend	Render (Web Service)	Comando de construcción: `npm install --include=dev && npm run build`. Comando de inicio: `npm start` (ejecuta `node dist/index.js`). Variables del backend definidas en el panel. Render admite WebSockets, por lo que Socket.io funciona sin configuración adicional.
Base de datos	MongoDB Atlas	Clúster en la nube con acceso autorizado desde Render y credenciales exclusivas para la aplicación.
Despliegue continuo	GitHub	Cada `git push` a la rama `main` dispara la construcción y el despliegue automáticos en Netlify y en Render.

> **Nota sobre el plan gratuito de Render:** el servicio se suspende tras un periodo de inactividad, por lo que la primera petición posterior puede tardar algunos segundos en responder mientras el servicio se reactiva.

12. Conclusiones

TutorGO integra en una sola plataforma los elementos que plantea el enunciado: autenticación híbrida con Google y JWT, búsqueda geoespacial con cálculo real de traslado, agendas libres de traslapes, suscripciones periódicas, chat en tiempo real, reseñas, un módulo de moderación con suspensión y apelación, y reportería analítica con gráficos dinámicos.

El uso de TypeScript en ambas capas reduce los errores y documenta los contratos entre el cliente y el servidor; SCSS, junto con Bootstrap, permite mantener una identidad visual coherente y un diseño responsivo; y la arquitectura modular, con rutas y componentes de responsabilidad única, facilita el mantenimiento y la incorporación de nuevas funciones. El despliegue en Netlify, Render y MongoDB Atlas con despliegue continuo deja la plataforma disponible en línea y fácil de actualizar.

12.1 Posibles mejoras futuras

- Restringir los orígenes permitidos de CORS y de Socket.io al dominio del frontend.
- Permitir la carga directa de la hoja de vida en formato PDF, además del enlace.
- Notificaciones por correo o *push* para solicitudes de clase y mensajes nuevos.
- Pruebas automatizadas unitarias y de integración para las reglas de agenda y de moderación.

Anexo A. Glosario

Término	Definición
SPA	*Single Page Application*: aplicación web que carga una sola página y actualiza su contenido dinámicamente.
JWT	*JSON Web Token*: token firmado que identifica la sesión del usuario y se envía en el encabezado `Authorization`.
OAuth 2.0	Protocolo de autorización que permite iniciar sesión con una cuenta de un tercero, en este caso Google.
WebSocket / Socket.io	Tecnología de comunicación bidireccional en tiempo real entre el navegador y el servidor.
Haversine	Fórmula que calcula la distancia en línea recta entre dos puntos de la superficie terrestre.
OSRM	*Open Source Routing Machine*: servicio que calcula rutas y distancias por calles.
ODM	*Object Document Mapper*: capa (Mongoose) que relaciona objetos del código con documentos de MongoDB.
SCSS	Sintaxis de Sass, un preprocesador que añade variables, mixins y anidamiento a CSS.
Mixin	Bloque de estilos reutilizable de SCSS que se incluye con `@include`.
Traslape	Choque de horarios entre dos clases de un mismo tutor o de un mismo estudiante.